

阅读文献笔记

No.

Date

阅读文献笔记

① ~~Prefill~~ Prefill: 处理输入, 建立输入的KV, 并得到首token的分布

② Decode: 用已有KV继续预测, 每步补入新token的KV

③ 流式返回可以边生成边进行, 不必等全部生成完

④ KV Cache

(i) 过去的token不会因后面生成而改变, 所以能保留KV

(ii) KV Cache 存各层 token KV 张量, 不是答案之本

(iii) 新token仍要计算注意力, 缓存不等于省掉所有计算

⑤ prefix reuse: 请求准备/调度时匹配前缀

让 prefill 少算命中部分; Decode 继续读取复用KV

⑥ paged-sized KV cache:

(i) 贯穿 Prefill, Decode 的存储管理

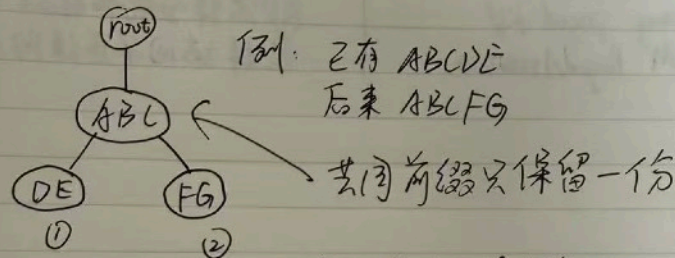
(ii) 负责KV的分配、寻址和回收, 不是一次额外模型推理。

前缀复用 → 哪些不用重复算

页管理 → KV 放在哪里, 怎样分配

⑦ Radix Attention

1) 实现: Radix tree



查找最长已缓存前缀 → 复用 →
计算后缀 → 插入前缀

请求结束后可保留缓存, 空间不足再淘汰
注意用 LRU 优先淘汰久未使用的可淘汰叶子。

相同子串中相同前缀, 在匹配开头连续
相同的 token 前缀。

⑧ Paged Attention

解决: 请求输出长度不固定预留大段连续 KV 空间浪费

Paged Attention 让注意力访问不连续的 KV 块。

+ 按需分配, 块表映射与块共享, 提高显存利用

"Paged"

L₀: [t1 t2 t3 t4]

L₁: [t5 t6 t7 t8]

L₂: [t9 t10 t11 t12]

物理块 P7

物理块 P8

物理块 P9

逻辑连续
物理不必连续

10 token → 3 块 → 未填满 2 个位置。

KOKUYO

机制	核心问题
Radix Attention / Radix Cache	哪段前缀 KV 能复用?
SGLang paged KV	KV 怎样分配和存放?
vLLM Paged Attention	怎样访问不连续的 KV 块

